CHAPTER 4

Wednesday: Blocks

Yesterday you learned a lot about methods and method calls. Today you will deal with *blocks*.

You're probably already familiar with blocks—you can't write much Ruby code without them. But what you might not know is that blocks are a powerful tool for controlling *scope*, meaning which variables and methods can be seen by which lines of code. In this chapter, you'll discover how this control of scope makes blocks a cornerstone of Ruby metaprogramming.

Blocks are just one member of a larger family of "callable objects," which include objects such as procs and lambdas. This chapter shows how you can use these and other callable objects to their greatest advantage—for example, to store a block and execute it later.

Just a short public service announcement before getting started: the previous chapters never strayed far from the usual object-oriented concepts, such as classes, objects, and methods. Blocks have a different heritage that can be traced back to functional programming languages, such as LISP. If you think in objects and classes, expect to deal with some novel concepts in this chapter. You're likely to find these concepts strange and, at the same time, fascinating.

With that sneak peek into what this chapter is all about, it's now time to step into the office.

The Day of the Blocks

Where you and Bill agree to put off today's job, make a roadmap, and review the basics of blocks.

You've barely had time to check your mail, and Bill is already making his way to your desk, eager to get to work. "I talked with the boss about today's job," he says. "I won't go into the details now, but I can tell you that we're going